

Defense guide: eval-churn-gate

For reading before an interview. Every number in this file is re-measured by `tools/collect_metrics.py` and checked against this text by `tools/check_numbers.py`, so a sentence that has gone stale fails the build rather than getting read aloud.

Read it in three passes. The first two sections are what to say in the first minute. The claims table and the sections under it are what to say when someone picks one. The last three sections are what to say when someone attacks it, which is the part worth rehearsing.

The thirty second version

"A regression gate on an aggregate score is a function of two numbers, and two numbers cannot tell you whether the same items improved or a different set improved and regressed in equal measure. I built a corpus where the true effect is known, including comparisons of a version against itself where it is exactly zero rather than approximately zero, and measured what each gate actually does. The gate almost everyone ships catches 0.323 of the regressions and misses both of the ones whose aggregate is about zero."

Then stop. The question that usually comes next is "so what would you ship instead", and the useful part of the answer is that it depends on which mistake costs more, and both sides are measured.

The two minute version

The structural claim first, because it is the one that does not depend on my corpus. The number of items whose outcome changed is a function of the *sum* of the two transition counts. The difference of means is a function of their *difference*. A sum and a difference are independent quantities, so a compensating regression is not a hard case for an aggregate gate, it is outside what that gate is a function of. Nothing about better statistics on top of the aggregate reaches it.

Then the measurement, because the structural claim alone does not tell anyone how much it costs them. The tool grades every gate against a truth the gates cannot see. On the scenario where one slice collapses, **a slice loses 0.1173 while the aggregate moves by 0.0006** in the median case, the mean threshold gate **sees it 0.0 of the time**, and the slice scan **catches it 1.0 of the time**. The combined gate **catches 1.0, at a false block rate of 0.037**, and that price is printed rather than buried.

Then the number I would lead with if I only had one. Over **1000 comparisons whose true effect** is exactly zero, run to run variation alone **flips 0.1167 of items** across the pass boundary, with a **ninety fifth percentile of 0.1467**, and **0.8889 of that movement** is cancelled by the net. One item in nine changes side while the average reports approximately nothing.

The claims, and how each one is proved

Claim	Command	The number that settles it
Three of five gates cannot see a regression that keeps the mean	<code>python experiments/exp02_the_regression_the_aggregate_cannot_see.py</code>	mean threshold 0.0, slice scan 1.0, on the same comparison
The aggregate carries no information about the churn	<code>python experiments/exp04_the_aggregate_carries_no_churn_information.py</code>	rank correlation 0.001 in magnitude, p value 0.963
Pairing is worth a factor of four and costs nothing	<code>python -m churngate power</code>	0.0122 paired against 0.0463 unpaired at 300 items
A threshold set at the regression size is a coin flip	<code>python experiments/exp03_what_the_eval_set_can_resolve.py</code>	0.455 against 0.99 for the paired interval
The gate publishes its own error rates	<code>python experiments/exp01_the_gate_graded_against_nothing.py</code>	0.037 false blocks, measured on an exact zero
A rate quoted without an interval is not a finding	<code>python -m churngate sweep</code>	0.084 at two hundred and fifty replications, 0.055 at three thousand

Three of five gates cannot see a compensating regression

One slice loses 0.1173 while the aggregate moves by 0.0006 in the median case. The mean threshold gate sees it 0.0 of the time. The paired tests fire at 0.022, against their own false block rate of about 0.04 on comparisons where nothing changed, so they are not detecting it, they are firing at their own noise rate. The slice scan catches it 1.0 of the time, and the reason is not cleverness: it tests a partition rather than a total.

If pushed on "so the slice scan is the answer": it misses pure churn, catching only 0.028 of it, while the churn gate **catches that one 1.0**. Two blind spots, two gates, and the combined gate is what covers both. Neither of them is the aggregate.

The impossibility has a number attached

Across **2133 comparisons whose delta** is within five thousandths of zero, the share of items that changed side runs **from 0.0667 to 0.5933 of items**, with a **rank correlation of 0.001 in magnitude** and a **p value of 0.963**. Restricted to the **2000 comparisons whose true aggregate** effect is exactly zero, the **correlation there is 0.0027 in magnitude**.

If pushed on "correlation is not the same as no information": correct, and that is why the structural argument comes first. The aggregate is a function of the difference between two transition counts and the churn is a function

of their sum, so the independence is a property of the arithmetic. The correlation is the measurement that confirms it, not the reason to believe it.

Pairing is worth a factor of four and costs nothing

At the shipped 300 item eval set the smallest detectable regression is **0.0122 paired**, against **0.0463** when two averages are compared. That is **worth a factor of 3.8** on the same data. Detecting the configured threshold **needs 112 items paired**, or **1608 comparing two averages**.

If pushed on "why does anyone compare two averages then": because the per item scores were averaged away before anything was written down. The cost of pairing is keeping them, which most pipelines already do in memory and then discard.

A threshold set at the regression size is a coin flip

At a true drop equal to the configured threshold, the mean threshold gate **catches it 0.455 of the time it happens**, against **0.99 for the paired** interval on the same data. The observed value lands either side of the threshold with roughly equal probability, so a gate set at the size of the thing it was built for is a coin flip on it.

If pushed on "then just lower the threshold": that raises the false block rate, and the point is that the paired interval gets the detection without the trade, because it uses the information the aggregate threw away.

The gate publishes its own error rates

The combined gate **catches 1.0**, at a **false block rate of 0.037** across the clean scenarios. The paired interval alone **blocks 0.039 of clean** comparisons, and the extra conditions are what buy the two scenarios it misses entirely. If blocking a good release is the expensive mistake instead, the churn gate **catches 0.384 overall** while **blocking only 0.005**.

If pushed on "your A/A comparisons are synthetic": they are, and that is exactly what makes the zero exact rather than approximate. A real A/A pair is only A/A if nothing else changed, which is the thing nobody can verify. The synthetic construction is not a shortcut here, it is the only way to get a true effect that is a fact rather than an estimate.

Latency, if anyone asks

The shipped configuration **decides in 14.9 ms** at the median, with a **p95 of 16.6 ms**, over **15 timed repeats** per size on a two vCPU container running **Python 3.11.15**. Measured **out to 20000 items** it is **707.7 ms at the largest size**, a **linearity ratio of 0.71** across the range. The paired bootstrap is **0.479 of the total cost** there and **allocates 305.2 MB** for its resample matrix, which is why **p95 rises to 1723.1 ms** and **p99 is 2.76 times p50** at the top of the table: an allocation profile rather than an arithmetic cost. The useful contrast is that the **slice scan costs 5.6 ms** on its own, **126 times cheaper than the total**, and it is the gate that catches the thing this repository is about, while the **mean threshold gate costs 0.015 ms** and catches none of it.

Questions that are meant to be hard

Is this just a paired t test? The paired test is one of five gates and one of the three that cannot see a compensating regression. What is mine: the transition decomposition with the net against gross split, the A/A anchor and the three quantities it makes measurable, the slice scan with a Holm correction and a measurement of what it costs, the third verdict, and the receipts pipeline that fails the build when a document quotes a number the code no longer produces. Around eight hundred statements of source, **207 tests**, **99.3 percent line coverage**.

Your scores are generated. Does any of it transfer? Two kinds of claim, and they transfer differently. The structural one, that an aggregate cannot bound the churn, is arithmetic and transfers everywhere. The magnitudes are properties of these noise parameters, a run to run standard deviation of 0.06 and an item spread of 0.22, and the tool reads a policy file precisely so a team can put their own measured numbers in it. Do not let this question get answered with "it transfers" alone.

What is the weakest part? The noise model has two components and reality has more. A real eval has run level effects that shift every item together, which pairing cannot remove and more items cannot shrink, and this model has no such term, so every minimum detectable effect here is the optimistic case. Second weakest: the slice scan only works because the losing group is attached to an observable label, and a regression confined to an unobservable subgroup is invisible to every gate here and to every gate anyone could write.

Why three verdicts rather than two? Because on comparisons whose true effect is exactly zero the honest answer is "this eval set cannot resolve a difference this small" **0.928 of the time the honest answer** is exactly that. This tool **blocked 0.069 of them** and **passed only 0.003**. A two outcome gate returns pass for all of them, and a pass is read as evidence of no regression when there was no evidence either way.

Why not statsmodels? Because the power arithmetic is the most quoted thing in the repository and it is four lines, so importing it would mean the headline number comes from a function a reviewer cannot check in thirty seconds. ADR-004 also records the honest version: if this were a production tool rather than a repository whose purpose is to show the reasoning, that trade would go the other way.

Did anything go wrong while you built it? Four things, and all four are in the README under "Hardest problem solved" with the fix beside each. The two worth telling out loud: the churn threshold was written from intuition at **first value was 0.06** while the measured floor is about twice that, so the gate blocked every comparison whose true effect was exactly zero, and the policy loader now refuses a limit below what the configured noise produces, with the shipped value at **a limit of 0.17**. And an early run measured a false block rate of 0.084 for a test whose true level is 0.05, at two hundred and fifty replications. It looked like a real finding for about an hour. At three thousand replications it was 0.055. That is why every rate now carries a Wilson interval and there is an explicit method for asking whether an interval excludes a value.

How do you know the noise model is doing what you say? Two arithmetic checks that are in the repository rather than in an argument: an observed **standard deviation of 0.0046** for the aggregate delta against the **predicted 0.0049** from the noise parameters alone. If those two disagreed, the whole power section would be decorative.

What this repository cannot establish

Its own section, because it is the part a senior reviewer reads first, and offering it before being asked is worth more than any of the claims above.

- **The scores are generated.** Two scenarios have an exact true effect, and that exactness is a property of the construction rather than of any dataset. What transfers is the method and the arithmetic.
- **The noise model is optimistic by construction.** No run level term, so the minimum detectable effects are the best case rather than the expected case.
- **The slice scan needs slices.** No amount of statistics recovers a partition nobody recorded.
- **The gates are graded on detection and false blocks, not on cost.** The exchange rate between missing a regression and blocking a good release is a property of a team, so the comparison ends in a table rather than a recommendation.
- **Nothing here is about answer quality.** A score drop is not the same as harm a user would notice. That is the next thing, and it is a different harness with a different labelling cost.

Things to say, and things not to say

Say:

- "the aggregate is a function of the difference and the churn is a function of the sum, so one cannot bound the other."
- "the A/A comparison makes the false block rate a measurement rather than an assumption."
- "the combined gate costs 0.037 on clean releases, and here is what that buys."
- "the minimum detectable effect here is the optimistic case, because the noise model has no run level term."
- "I do not know which mistake costs more in your pipeline, so I measured both."

Do not say:

- **"zero false blocks."** Say the mean threshold gate blocked none, which supports **below 0.05 percent over 2000 clean releases** and nothing stronger.
- **"the paired t test runs hot."** Its measured false block rate is not distinguishable from the nominal level at ninety five percent confidence, and an early run of this suite made exactly that mistake in front of nobody, which was lucky.
- **"chunking the metric differently fixes it."** A pass rate is the net of the transitions, so it is precisely the quantity that cancels compensating movement.
- **"99.3 percent coverage means it is correct."** It means the lines ran. The A/A anchor is the thing that means something.
- **"the gate catches everything."** It catches everything in this corpus, which was built by me, and the sentence has to carry that clause.

The live demo, five commands

```
# 1. What this eval set can resolve, before running any comparison.
python -m churngate power

# 2. A slice collapses and the aggregate does not move. Exit 1.
python -m churngate gate --scenario compensating

# 3. The gate almost everyone ships, on the same comparison. It passes.
python -m churngate one --gate mean_threshold --scenario compensating

# 4. Every gate on every scenario, with an interval on every rate.
python -m churngate sweep --replications 200

# 5. Every published figure re-measured and checked against the documents.
make receipts
```

If there is time for one more, make bench regenerates the latency table and make charts redraws the chart from its JSON, which is the shortest demonstration that no figure in either document was typed by hand.

Where to look in the code

Question	File
What makes a true effect exact rather than estimated	src/churngate/runs.py, the scenario constructions
The four transition cells and the net against gross split	src/churngate/churn.py
The five gates, the power arithmetic, the churn floor	src/churngate/gates.py
Why a rate cannot be quoted without its interval	src/churngate/rates.py
The three verdicts and the exit codes they carry	src/churngate/audit.py
Every threshold, with a comment on who chose it and how	configs/policy.yaml
The five decisions and their rejected alternatives	docs/adr/